

ONIRAJA

Product Requirements Document

Commerce and Operations Platform for Nigerian Merchants

Product	Oniraja Commerce and Operations Platform
Repository	OnirajaPOS
Version	2.0
Status	Active development; implementation-aligned baseline
Date	September 2026
Owner	Product and Engineering
Primary market	Nigerian small and medium-sized merchants
Currency at launch	NGN, stored as integer kobo

Contents

1. Document Purpose	3
2. Product Summary	3
3. Problem Statement	3
4. Goals and Success Metrics	4
5. Personas and Roles	4
6. Product Principles	5
7. Tenant and Account Model	6
8. Authentication, Authorization, and Security	6
9. Merchant Onboarding	7
10. Catalogue and Product Management	8
11. Inventory and Stock Control	8
12. Orders and Fulfilment Pipeline	9
13. Payments and Double-Entry Ledger	9
14. POS	10
15. Customer Storefront	11
16. WhatsApp Commerce	11
17. Customers and Consent	12
18. Expenses, Invoicing, and Documents	12
19. Reporting and Financial Intelligence	13
20. Shipping, Pickup, and Logistics	14
21. Staff, Locations, and Business Settings	14
22. Subscriptions and Feature Entitlements	15
23. Platform Operations Console	15
24. Notifications and Background Processing	16
25. Observability and Operational Controls	17
26. Non-Functional Requirements	17
27. Release Scope and Definition of Done	18
28. Explicitly Deferred or Out of Scope	19
29. Risks and Mitigations	19
30. Traceability and Source Documents	19

1. Document Purpose

This is the canonical product requirements document for the Oniraja platform. It translates the current backend modules, companion frontend plans, and operational requirements into a single product view.

Each feature is marked with an implementation state:

- **Implemented:** present in the backend or supporting infrastructure.
- **In progress:** partially implemented or awaiting integration/hardening.
- **Planned:** required product behavior that is not yet complete.
- **Deferred:** intentionally excluded from the current release.

The API contract in generated OpenAPI documentation is authoritative for exact request and response field names. This document defines product behavior, user intent, boundaries, and acceptance criteria.

2. Product Summary

Oniraja is a multi-tenant commerce and business operations platform for Nigerian merchants. It unifies catalogue, inventory, orders, payments, accounting, POS, online storefront, WhatsApp commerce, logistics, staff, subscriptions, and reporting.

The product's central promise is one reliable operational and financial record for every sale, regardless of where the sale began. A merchant can sell from a storefront, POS, WhatsApp, or a manually created order while Oniraja keeps stock, payment state, fulfilment, ledger entries, and reporting synchronized.

Oniraja is differentiated by:

1. A shared order pipeline across all sales channels.
2. Double-entry financial records instead of sales-only totals.
3. Schema-per-tenant isolation for merchant data.
4. Nigerian payment, phone, state, delivery, and consent workflows.
5. A platform operations plane for secure cross-tenant support and administration.

3. Problem Statement

Nigerian SMEs commonly operate with disconnected tools: spreadsheets for stock, a payment gateway dashboard for collections, WhatsApp for customer conversations, a separate POS, and manual bookkeeping. This creates operational and financial failure modes:

- Orders are missed or duplicated across channels.
- Stock is inaccurate and overselling is common.
- Payment confirmation does not reliably update fulfilment.
- Owners cannot trust profit figures or reconcile refunds and expenses.

- Branches and staff cannot be managed consistently.
- Support teams cannot diagnose tenant issues without unsafe database access.

Oniraja provides a single, auditable system that turns each commercial event into synchronized inventory, order, payment, ledger, notification, and reporting data.

4. Goals and Success Metrics

4.1 Launch goals

Goal	Measure	Target
Merchant activation	Merchant reaches first completed sale	Within one onboarding session
Channel coverage	Active selling channels	Website and POS minimum; WhatsApp when Meta approval is live
Financial integrity	Balanced journal entries	100% of posted entries
Payment reliability	Duplicate webhook side effects	Zero
API performance	Read endpoint p95 latency	Under 300 ms under normal load
Platform reliability	Normal-load 5xx rate	Under 1%
Subscription operation	Automatic renewal	No manual intervention for normal renewals
Support safety	Platform mutations with audit records	100%

4.2 Product outcomes

- Merchants can know what they sold, what remains, and whether they made money.
- Customers can discover and purchase products through the channel they prefer.
- Staff can operate within clear role and location boundaries.
- Internal support can resolve common issues without unrestricted tenant access.

5. Personas and Roles

5.1 Merchant owner

Owns the business, controls financial data, manages staff and settings, and needs cross-location visibility.

As a merchant owner, I want a trustworthy view of revenue, profit, stock, and cash so that I can make business decisions without reconstructing them from several tools.

5.2 Manager

Runs day-to-day operations for one or more locations and needs to manage products, inventory, orders, and staff within delegated permissions.

As a manager, I want to process sales, correct stock, and monitor my location so that the business keeps operating without requiring the owner for every action.

5.3 Staff or cashier

Processes POS sales, handles customers, and performs assigned operational tasks.

As a cashier, I want to complete a sale quickly and record the correct payment method so that customers are not delayed and the sale is attributed to me.

5.4 Viewer

Needs read-only operational or financial visibility without mutation privileges.

As a viewer, I want to inspect permitted reports without being able to change business data.

5.5 Customer

Browses a merchant catalogue and places orders through a web storefront, POS-assisted sale, or WhatsApp conversation.

As a customer, I want clear products, totals, delivery choices, payment, and order updates so that I can buy with confidence.

5.6 Platform support

Works across tenants to inspect issues and perform a fixed set of audited repair actions.

As support, I want time-boxed access to a tenant's relevant data so that I can resolve an issue without unrestricted access to every merchant.

5.7 Platform admin and super-admin

Manage tenant lifecycle, plans, platform users, permissions, audits, and operational health.

As a platform admin, I want aggregate tenant and billing visibility so that I can operate the platform proactively.

6. Product Principles

- **One source of truth:** all channels use the same product, order, payment, inventory, and ledger concepts.
- **Server authority:** totals, permissions, transitions, and entitlements are enforced on the server.
- **Auditability:** every stock, money, access, and administrative mutation has an attributable record.
- **Least privilege:** users see and change only what their role, location, plan, and support grant allow.
- **Async where appropriate:** notifications, image processing, billing, logistics, and low-stock work run through retryable jobs.
- **Nigerian by default:** NGN/kobo, Nigerian states, local phone formats, Paystack/Flutterwave, NDPR consent, and Africa/Lagos timezone support are first-class.

7. Tenant and Account Model

Requirements

- **R7.1** Each merchant is a tenant with an isolated PostgreSQL schema.
- **R7.2** Public data contains tenant identity, billing, integrations, and platform records; tenant data contains users, products, orders, payments, and operational records.
- **R7.3** Tenant context is resolved before tenant-scoped handlers execute.
- **R7.4** A tenant can have multiple locations, users, products, channels, and subscription states.
- **R7.5** Deactivated tenants and users cannot authenticate or perform operations.

User stories

- As a merchant, I want my business data isolated from other merchants so that my customers and finances remain private.
- As the platform, I want tenant provisioning to create the schema, migrations, defaults, and ledger accounts atomically so that every new merchant starts in a valid state.
- As a merchant, I want a trial account so that I can validate the product before subscribing.

Acceptance criteria

- Tenant creation provisions the tenant schema and required default records.
- Tenant-scoped access cannot select another tenant by changing a request parameter.
- Provisioning failure does not leave an apparently active, unusable tenant.

- Default asset, liability, equity, revenue, and expense accounts exist before financial transactions can post.

8. Authentication, Authorization, and Security

Tenant authentication

- Login, refresh, logout, current-user lookup, password reset, and invite acceptance.
- Short-lived access credentials with refresh-token rotation and revocation.
- Passwords are hashed with Argon2.
- Tenant roles are `owner`, `manager`, `staff`, and `viewer`.

Platform authentication

Platform users are separate from tenant users. The platform plane uses separate identity records, JWT namespace/audience, secret, sessions, permissions, and audit records.

- Platform roles: `read_only`, `support`, `admin`, `super_admin`.
- TOTP MFA is required for high-privilege platform accounts and should be enforced for all platform users.
- Platform access tokens contain no tenant ID.
- Platform sessions are short-lived and revocable.

User stories

- As an owner, I want to invite staff with a specific role so that access matches responsibility.
- As an owner, I want to deactivate a departing staff member immediately without deleting historical records.
- As a platform user, I want MFA so that a stolen password alone cannot expose every tenant.
- As a support user, I want a permissioned support grant so that my access is limited in scope and time.

Acceptance criteria

- Permission checks run server-side on every protected route; UI hiding is not a security control.
- `viewer` cannot mutate operational or financial data.
- Role hierarchy is enforced consistently by middleware.
- All PII reveal, support access, platform mutations, and authentication-sensitive events are audited.
- Browser clients do not store platform or tenant access tokens in JavaScript-readable storage.

9. Merchant Onboarding

Implementation state: Implemented foundation; integration and completion checks remain in progress.

Scope

Guided onboarding captures business identity, primary location, owner account, initial catalogue, operating preferences, and trial setup. The flow should end with a merchant able to create a first order.

User stories

- As a new merchant, I want guided setup so that I can reach my first sale without understanding the entire system.
- As a merchant, I want to add my first products and location during setup so that the dashboard is useful immediately.
- As the platform, I want onboarding progress to be queryable so that support can identify where activation stopped.

Acceptance criteria

- Onboarding progress is resumable and idempotent.
- Required business and owner fields are validated before completion.
- A completed onboarding has an active location, usable product or catalogue state, owner access, and trial/subscription state.
- Onboarding status is visible to authorized platform operations users.

10. Catalogue and Product Management

Implementation state: Implemented.

Scope

Products support hierarchical categories, active/inactive state, descriptions, images, variants, SKUs, prices, costs, tax rates, attributes, weight, and low-stock thresholds.

User stories

- As an owner, I want to create a product with variants such as size or color so that my catalogue matches what I sell.
- As a manager, I want to update price, cost, SKU, and availability so that the storefront and POS stay current.
- As a merchant, I want uploaded images compressed and stored reliably so that product pages load quickly.

- As a customer, I want inactive or unavailable products hidden or marked out of stock so that I cannot buy something the merchant cannot fulfil.

Acceptance criteria

- Each variant has a unique merchant-scoped SKU, price in kobo, optional cost in kobo, tax basis points, attributes, and optional weight.
- Products and variants can be soft-deactivated without removing historical order lines.
- Image upload validates type/size, compresses server-side, and returns durable object storage metadata.
- Cost data is restricted to authorized merchant roles.
- Category filters support parent/child relationships.

11. Inventory and Stock Control

Implementation state: Implemented; load and integration validation remain in progress.

Scope

Inventory is tracked per variant and location. The system supports receiving, sales, returns, adjustments, transfers, movement history, availability checks, and low-stock alerts.

User stories

- As a manager, I want to see stock at my location so that I know what can be sold now.
- As a manager, I want to receive stock from a supplier so that on-hand quantities reflect deliveries.
- As an owner, I want every adjustment to require a reason so that shrinkage and corrections are explainable.
- As a merchant, I want low-stock notifications so that I can reorder before a stockout.
- As a merchant, I want a return or cancellation to restock the correct location so that stock remains accurate.

Acceptance criteria

- Stock is keyed by variant and location.
- Every movement records type, quantity, reference, actor, location, and timestamp.
- Quantity cannot become invalid through concurrent or repeated operations.
- Sale, return, cancellation, transfer, receive, and adjustment movements are distinguishable.
- Low-stock checks run asynchronously and do not block checkout unnecessarily.
- Inventory valuation is quantity multiplied by variant cost and is available to authorized reporting users.

12. Orders and Fulfilment Pipeline

Implementation state: Implemented.

Scope

Every sale is an order with a channel, customer, line items, location, totals, payment state, fulfilment state, and audit history. Channels are `website`, `pos`, `whatsapp`, and `manual`.

Order state is controlled by a state machine. Payment state is separate from order state.

User stories

- As a cashier, I want to create a walk-in order quickly so that every sale is recorded.
- As a merchant, I want website, POS, WhatsApp, and manual orders in one list so that fulfilment has one workflow.
- As a merchant, I want payment confirmation to advance the order appropriately so that webhooks do not require manual reconciliation.
- As a manager, I want cancellation and refund behavior to update inventory and finance correctly so that records remain consistent.
- As a customer, I want an order number and status updates so that I know what happens after checkout.

Acceptance criteria

- Server calculates subtotal, discount, tax, delivery fee, and total; client totals are never trusted.
- Invalid state transitions return a validation error and do not mutate data.
- Payment status and order status can change independently where the workflow requires it.
- Order line items preserve product/variant name and price snapshots for historical accuracy.
- Cancellation, return, and refund behavior is idempotent.
- Orders identify their source channel, location, assigned staff member, and customer where available.

13. Payments and Double-Entry Ledger

Implementation state: Implemented; end-to-end integration coverage is planned.

Scope

Paystack and Flutterwave are accessed through a gateway abstraction. Payments support initiation, verification, webhook processing, refunds, and methods such as card, transfer, USSD, and cash/manual payment where applicable.

The ledger contains accounts, journal entries, and journal lines. Payments, refunds, and expenses post balanced entries.

User stories

- As a customer, I want to pay by card, transfer, or USSD so that I can choose a method I trust.
- As a merchant, I want a successful payment to reconcile automatically so that I do not manually update accounts.
- As a merchant, I want a refund to reverse the original financial impact so that reports remain truthful.
- As the platform, I want webhook events deduplicated so that retries never double-count money.
- As an owner, I want a ledger-backed P&L so that profitability is based on accounting entries rather than a rough order sum.

Acceptance criteria

- All monetary values remain integer kobo through storage, transport, and calculation.
- Webhook signatures are verified before processing.
- Gateway event IDs are unique and duplicate events are no-ops.
- Journal entries commit only when total debits equal total credits.
- Payment and ledger mutations are transactional.
- Gateway failures expose actionable state without falsely marking an order paid.
- Refunds create auditable reversing entries and cannot be applied twice.

14. POS

Implementation state: Backend implemented; frontend delivery in progress/planned.

Scope

A full-screen, fast checkout surface for in-person transactions. It supports product search, barcode-ready lookup, cart editing, discounts, customer lookup/creation, cash/transfer/card/USSD/manual payment, split-payment requirements where supported, receipts, and cashier attribution.

User stories

- As a cashier, I want product search and one-tap cart addition so that checkout is fast.
- As a cashier, I want to apply an authorized discount so that I can honor approved customer arrangements.
- As a cashier, I want to record cash, transfer, or card payment accurately so that the till reconciles.
- As a merchant owner, I want each POS order tied to the cashier and location so that performance and accountability are visible.

- As a customer, I want a receipt that can be printed or shared so that I have proof of purchase.

Acceptance criteria

- POS orders are created with `channel = pos`.
- The logged-in staff member and location are captured.
- POS has no distracting dashboard navigation and has an explicit exit path.
- Payment completion is not reported until the selected payment method is recorded successfully.
- Unauthorized discounts, refunds, and stock actions are blocked server-side.

15. Customer Storefront

Implementation state: Backend support implemented; frontend planned/in progress.

Scope

A public merchant storefront supports catalogue browsing, category filtering, product details, variant selection, cart, delivery address, shipping selection, payment redirect, confirmation, and order tracking.

User stories

- As a customer, I want to browse a merchant's active products by category so that I can discover what is available.
- As a customer, I want variant-specific price and availability so that I order the exact item I want.
- As a customer, I want to see delivery methods and fees before paying so that there are no surprises.
- As a customer, I want hosted payment checkout so that my payment details are handled by a trusted gateway.
- As a merchant, I want unavailable variants excluded from checkout so that I do not accept unfulfillable orders.

Acceptance criteria

- Public catalogue responses contain only active products and variants.
- Checkout revalidates price, stock, customer data, consent, and delivery fee server-side.
- Storefront orders use `channel = website`.
- NDPR consent records timestamp and source at data collection.
- Payment success and failure return clear, recoverable outcomes.

16. WhatsApp Commerce

Implementation state: Scaffold and session state machine implemented; live integration pending Meta approval.

Scope

A WhatsApp flow lets a customer greet the merchant, browse a catalogue, add items to a session cart, review totals, receive payment instructions, and create a standard order.

User stories

- As a customer, I want to start with a greeting or menu so that I do not need to learn commands.
- As a customer, I want a running cart total inside WhatsApp so that I can confirm before paying.
- As a customer, I want a payment link in the conversation so that I can finish without navigating an unfamiliar site.
- As a merchant, I want WhatsApp orders in the same order queue as every other channel.
- As the platform, I want abandoned sessions to expire so that stale carts do not accumulate.

Acceptance criteria

- Meta webhook signatures are verified.
- Redis-backed session state uses a TTL and the states `idle`, `browsing`, `cart`, `checkout`, `awaiting_payment`, and `complete`.
- Completed orders use `channel = whatsapp` and the standard order/payment pipeline.
- Missing WhatsApp credentials disable the integration cleanly rather than breaking unrelated channels.
- Live interactive catalogue and payment-link behavior is covered by integration tests when credentials are available.

17. Customers and Consent

Implementation state: Implemented.

User stories

- As a merchant, I want a searchable customer directory and purchase history so that I can serve repeat customers better.
- As a customer, I want my data collected transparently so that I understand how it is used.
- As a merchant, I want customer details attached to orders across channels so that service history is unified.

Acceptance criteria

- Customer creation and update validate Nigerian-compatible contact data.
- Consent captures source and timestamp for storefront, POS, and WhatsApp collection.
- Customer records can be updated without breaking historical orders.
- PII is not exposed to roles that do not need it.

18. Expenses, Invoicing, and Documents

Implementation state: Implemented; delivery hardening remains in progress.

User stories

- As an owner, I want to record an expense with category, amount, date, and receipt so that every business cost is documented.
- As an owner, I want expenses posted to the ledger automatically so that P&L stays current.
- As a merchant, I want to generate an invoice from an order so that customers receive a formal record.
- As a merchant, I want to send an invoice by email and retain a downloadable PDF so that the document is easy to share.

Acceptance criteria

- Expenses store amount in kobo, category, description, date, actor, and optional receipt URL.
- Expense creation posts a balanced journal entry.
- Invoice lifecycle supports **draft**, **sent**, and **paid** with valid transitions.
- PDFs are generated and stored through signed object-storage URLs.
- Missing email configuration does not break invoice creation; delivery failure is observable.

19. Reporting and Financial Intelligence

Implementation state: Implemented; load validation remains in progress.

Scope

Reports include P&L, revenue by location, staff sales, inventory valuation, ledger views, wallet/reconciliation data, CSV exports, and dashboard KPI aggregates.

User stories

- As an owner, I want P&L by date range so that I can understand profitability.
- As an owner, I want revenue by location so that I can compare branches.

- As an owner, I want staff sales totals and transaction counts so that I can coach and reward the team.
- As an accountant, I want CSV exports so that I can continue analysis outside Oniraja.
- As a manager, I want inventory valuation so that I understand capital tied up in stock.

Acceptance criteria

- P&L derives revenue and expense values from ledger accounts.
- Date filters use explicit timezone-aware boundaries.
- Reports enforce role and feature permissions.
- Export results match the filtered on-screen dataset.
- Monetary responses expose integer kobo; clients format NGN only at presentation time.
- Platform-wide analytics use aggregate/rollup data and never scan every tenant schema per request.

20. Shipping, Pickup, and Logistics

Implementation state: Implemented; provider and live dispatch validation remain in progress.

Scope

Merchants configure flat, zone, value, weight, automated, free, and pickup methods. They define Nigerian shipping zones, rates, free-shipping conditions, merchant branches, and third-party collection points. Logistics integrations support quotes, dispatch, tracking events, and provider abstraction.

User stories

- As a merchant, I want different rates by Nigerian state or zone so that delivery reflects actual cost.
- As a merchant, I want free delivery above a threshold or with a promo condition so that I can run promotions.
- As a customer, I want available methods, fees, pickup points, and estimated delivery information before payment.
- As a merchant, I want a live provider quote before dispatch so that the fee is realistic.
- As a merchant, I want to trigger dispatch after packaging so that fulfilment timing remains under my control.

Acceptance criteria

- Shipping availability returns applicable methods sorted by fee with all required checkout context.
- Catch-all rate behavior is explicit when no zone matches.
- Order stores shipping method, pickup location, destination state, and delivery fee in kobo.
- Weight-based methods use variant weight and reject missing data when required.

- Provider credentials are encrypted at rest.
- Logistics webhook events are signature-checked where supported and idempotent by event ID.
- Dispatch status updates are mapped without corrupting the core order state machine.

21. Staff, Locations, and Business Settings

Implementation state: Staff and locations implemented; broader settings planned.

User stories

- As an owner, I want to create and manage locations so that each branch has its own stock and sales context.
- As an owner, I want to assign staff to locations so that operational views are appropriately scoped.
- As an owner, I want to edit business profile information so that storefront and documents remain accurate.
- As an owner, I want to see active sessions and revoke one so that I can respond to a lost device.
- As an owner, I want to export my business data so that I retain control of it under NDPR.

Acceptance criteria

- Location CRUD supports default location semantics.
- User location assignment is enforced in queries and mutations.
- Business settings update validates fields and records an audit event.
- Session revocation invalidates the selected refresh/session record.
- Data export is authenticated, scoped to the tenant, and delivered through an expiring link.

22. Subscriptions and Feature Entitlements

Implementation state: Implemented; operational hardening remains in progress.

Scope

Plans are **trial**, **entry**, **growth**, and **enterprise**. Subscription state is **trial**, **active**, **grace**, **lapsed**, or **cancelled**. Entitlements are configuration-driven.

User stories

- As a new merchant, I want a trial so that I can evaluate Oniraja without immediate commitment.
- As a paid merchant, I want recurring billing so that my account stays active without manual renewal.

- As a merchant with a failed charge, I want a grace period so that I can fix payment without immediate service loss.
- As the platform, I want entitlement rules in configuration so that plan changes do not require scattered code edits.

Acceptance criteria

- Subscription transitions are validated by a state machine.
- Failed charges enter grace before lapsed according to configured timing.
- Recurring billing uses the authorized gateway method and records outcomes.
- Feature gates return a consistent payment/entitlement error and do not rely on frontend-only checks.
- Subscription jobs are retryable and idempotent.

23. Platform Operations Console

Implementation state: Platform foundation and support tooling shipped; owner administration and analytics are planned.

Scope

The platform plane serves internal users across all tenants. It includes secure platform auth, tenant search/detail, lifecycle actions, support grants, narrow repair actions, billing visibility, platform user management, audit logs, and health/analytics.

User stories

- As support, I want to search for a tenant and view masked operational details so that I can diagnose an issue.
- As support, I want a read-only, time-boxed access grant with a reason so that tenant access is controlled.
- As an admin, I want to suspend/reactivate a tenant and change plan state so that account lifecycle is manageable.
- As a super-admin, I want to create or deactivate platform users so that internal access is governed.
- As an auditor, I want immutable records of actor, target, reason, before/after data, IP, and request ID so that sensitive actions are reviewable.
- As an operator, I want tenant health, queue, billing, and error visibility so that failures are found before merchants report them.

Acceptance criteria

- Platform routes require platform auth and explicit platform permission.

- Tenant access grants have scope, reason, expiry, revocation, and merchant-visible audit/notification behavior.
- Support repair actions are a fixed allowlist and cannot become arbitrary tenant mutations.
- PII is masked by default; reveal is separately permissioned and audited.
- Tenant suspension is honored by tenant authentication and request resolution.
- Platform audit is append-only; there is no update/delete path.
- Cross-tenant overview metrics use public aggregates or rollups rather than $O(N)$ schema iteration.

24. Notifications and Background Processing

Implementation state: Infrastructure implemented; provider and failure-path testing in progress.

Scope

BullMQ workers process payments, notifications, documents, inventory, logistics, and subscriptions. Notifications may use SMS, email, WhatsApp, and Slack depending on event and configuration.

User stories

- As a merchant, I want low-stock, payment, subscription, and order notifications so that important events are not hidden in the dashboard.
- As an operator, I want failed jobs retried and surfaced so that transient provider outages do not silently lose work.
- As an operator, I want Slack alerts for production errors so that response begins quickly.

Acceptance criteria

- Jobs have stable idempotency keys where a retry could duplicate an external side effect.
- Failed jobs are retryable and observable.
- Provider absence is handled explicitly and does not crash unrelated flows.
- Notifications do not expose more PII than the recipient and channel require.
- Graceful shutdown drains or safely requeues active work.

25. Observability and Operational Controls

Implementation state: Core instrumentation implemented; hardening planned.

Requirements

- Every request receives and returns an **X-Request-ID**.
- Structured logs include request, tenant, actor, route, status, and duration where applicable.

- Prometheus-compatible metrics expose request counts, latency, errors, queue health, and worker health.
- Sentry or equivalent captures unexpected exceptions.
- Health checks report application, database, Redis, and queue readiness.
- Rate limits apply to authentication, webhooks, public routes, and sensitive mutations.
- PostgreSQL backups and restore testing are mandatory before production sign-off.

26. Non-Functional Requirements

Security and privacy

- Follow OWASP guidance and least privilege.
- Encrypt secrets and provider credentials at rest.
- Use HTTPS in deployed environments.
- Apply NDPR principles: purpose limitation, consent capture, access control, export, and retention decisions.
- Never log passwords, tokens, full payment credentials, or unnecessary PII.

Reliability and consistency

- Financial, stock, and order mutations are transactional.
- Webhooks and workers are idempotent.
- External provider failures produce recoverable states.
- Historical records remain readable after catalogue or staff changes.

Performance and scale

- Tenant-scoped common reads target p95 under 300 ms under normal load.
- Pagination is required for potentially unbounded lists.
- Search and filters execute server-side.
- Platform aggregation uses denormalized metrics or rollups.

Localization

- NGN is the launch currency; monetary storage uses kobo integers.
- Default timezone is Africa/Lagos; timestamps are persisted unambiguously.
- Nigerian states are validated from a shared list.
- Phone and address forms support local Nigerian formats.

Accessibility and frontend quality

- Responsive web surfaces support desktop, tablet, and mobile workflows.
- Forms expose labels, errors, focus states, keyboard operation, and loading/success/error/empty states.
- POS interactions meet mobile tap-target requirements and minimize unnecessary navigation.
- BFF proxying keeps browser clients from calling the backend directly or holding tokens.

27. Release Scope and Definition of Done

Release 1: operational commerce core

Must include:

- Tenant provisioning, auth, RBAC, onboarding, and locations.
- Products, variants, categories, inventory, customers, and orders.
- POS and storefront purchase paths.
- At least one production payment gateway with verified webhooks.
- Balanced ledger entries for payments, refunds, and expenses.
- Subscription trial, billing, grace, and feature gating.
- Core reports, invoices, shipping configuration, and dispatch foundation.
- Observability, backups, restore test, and security review.

WhatsApp is included when Meta approval and credentials are available; otherwise it is formally deferred with the sandbox flow tested.

Release 2: platform maturity

- Complete live WhatsApp catalogue and payment flow.
- Complete settings, session management, NDPR export, and tenant audit writes.
- Platform user management, tenant health, billing views, and analytics rollups.
- Integration and load tests for payment-to-ledger, WhatsApp sessions, logistics, and multi-tenant queries.

Definition of done

A release is complete only when:

- A real merchant can onboard and make a sale.
- At least two channels create orders in the shared pipeline.
- Payment, inventory, order, and ledger outcomes reconcile.
- Role, tenant, plan, and support boundaries are tested server-side.

- Critical flows have automated tests and documented manual QA.
- Monitoring, backup, restore, and incident procedures are operational.

28. Explicitly Deferred or Out of Scope

- Virtual account issuance without a licensed partner.
- Holding customer funds or operating a regulated wallet without the required licensing.
- Multi-currency settlement; launch is NGN-only.
- FIRS e-invoicing or tax filing integration until the compliance pathway is confirmed.
- Loyalty, gift cards, advanced BI, and wholesale pricing tiers.
- Native mobile applications; responsive web is the initial mobile surface.
- Full internal ticketing; support lookup, grants, repair actions, and audit are the initial support scope.
- Business-owner features inside the platform operations console.

29. Risks and Mitigations

Risk	Impact	Mitigation
Payment webhook failure	Incorrect order and ledger state	Signature validation, unique event IDs, retries, alerts, reconciliation view
Cross-tenant access defect	System-wide data exposure	Separate auth plane, grant middleware, permission tests, masked PII, audit review
Schema iteration at scale	Slow platform dashboards	Public aggregates and rollups; never loop every tenant on each request
WhatsApp approval delay	Delayed differentiating channel	Build sandbox/state machine first; formally gate live launch on Meta approval
Provider outage	Delayed payments, documents, or dispatch	Retryable jobs, explicit pending states, operator alerts, manual recovery actions
Scope growth	Missed launch target	Treat this document's release boundaries as change-controlled scope
Missing backups or restore validation	Irrecoverable business data	Configure automated backups and perform a witnessed restore before production

30. Traceability and Source Documents

This PRD consolidates requirements from:

- [OnirajaPOS/PRD.md](#) - original product scope, personas, feature requirements, and build status.
- [OnirajaPOS/docs/ADMIN-PORTAL-PLAN.md](#) - platform auth, support grants, platform RBAC, audit, and admin phases.
- [sabby-client/docs/IMPLEMENTATION_PLAN.md](#) - frontend surfaces, API/BFF boundaries, role gating, and module sequencing.
- [sabby-client/docs/UI-DESIGN-BRIEF.md](#) - storefront, dashboard, POS, WhatsApp, localization, and required UI states.
- [sabby-admin/docs/ADMIN-DASHBOARD-PLAN.md](#) - platform console security posture and operational screen scope.

When implementation and this document diverge, update this PRD or record the decision in the relevant implementation plan. Do not silently change behavior in code without updating the acceptance criteria and tests.